Lecture 6 | MSc Data Handling and Infrastructure for AI

Data Pipelines: ETL, ELT and Google Cloud Dataflow

Building a real pipeline for Dublin Bikes data

Dr Abdul Wahid | SETU | 2026

Beam
write the pipeline once

Dataflow
run it on Google Cloud

BigQuery
ask questions of the result

1 Why pipelines?

Real organisations, and real messy open data from Dublin.

What is a data pipeline?

A data pipeline is an automated, repeatable series of steps that moves data from where it is produced to where it is used, checking and transforming it on the way.

Raw data in
files, feeds, databases

›

Automated steps
check, clean, combine, summarise

›

Trusted data out
ready for people and models

Think of a factory line

Raw material arrives, inspectors reject faulty parts and write down why, machines assemble the rest, and a finished product leaves, every day.

In our demo

Bike-station readings go in. Each row is checked, bad rows are set aside with a reason, good rows become hourly averages. A results table comes out.

A pipeline is a production line for data: raw material in, inspected and processed in stages, a reliable product out

The five jobs of a data pipeline



Collect

Get the data from where it lives



Validate

Check it against the contract



Transform

Clean, enrich, aggregate



Load

Put it where people can use it



Repeat

Do it again tomorrow, safely

Lecture 5 covered Validate and Transform as ideas. Today we make all five happen, automatically.

Doing the five jobs once is analysis; doing them reliably every time is engineering

Case 1: Spotify and the biggest Dataflow job

5×

Wrapped 2019 was five times larger than Wrapped 2018

¾

of the cost of the previous year's job

What Spotify did

Chose Google Cloud for its data infrastructure.

Collected and forwarded all events with Cloud Pub/Sub.

Used Dataflow to power its ETL workloads.

Open-sourced Scio in 2015: a Scala API for Apache Beam and Dataflow.

At this scale, cost and reliability become the design problem.

Sources: TechCrunch on Wrapped 2019; Tech Monitor and Google Cloud blog on Spotify's move. Company-reported figures.

Case 2: Palo Alto Networks and 30,000 pipelines

30,000

separate pipelines, one per tenant, before the change

~30%

compute cost saving after consolidating (vendor-reported)

What changed

Firewall logs, telemetry and threat events arrive every second.

One pipeline per customer meant thousands of jobs to run and watch.

They moved to a unified multi-tenant Dataflow pipeline.

Built-in logging and metrics now monitor pipeline health.

The lesson: fewer, well-designed pipelines beat many copies.

Source: Google Cloud blog, "How Palo Alto Networks built a multi tenant scalable Unified Data Platform". Vendor-reported figures.

Our case: Dublin Bikes open data

The data

Published by Dublin City Council, licence CC BY 4.0.

A live feed of every station, updated every minute.

Monthly history files of station status, 2018 to June 2026.

A small reference file with station number, name and location.

Used by real apps, and free for you to use.

40–80 MB

per monthly file

300k–635k

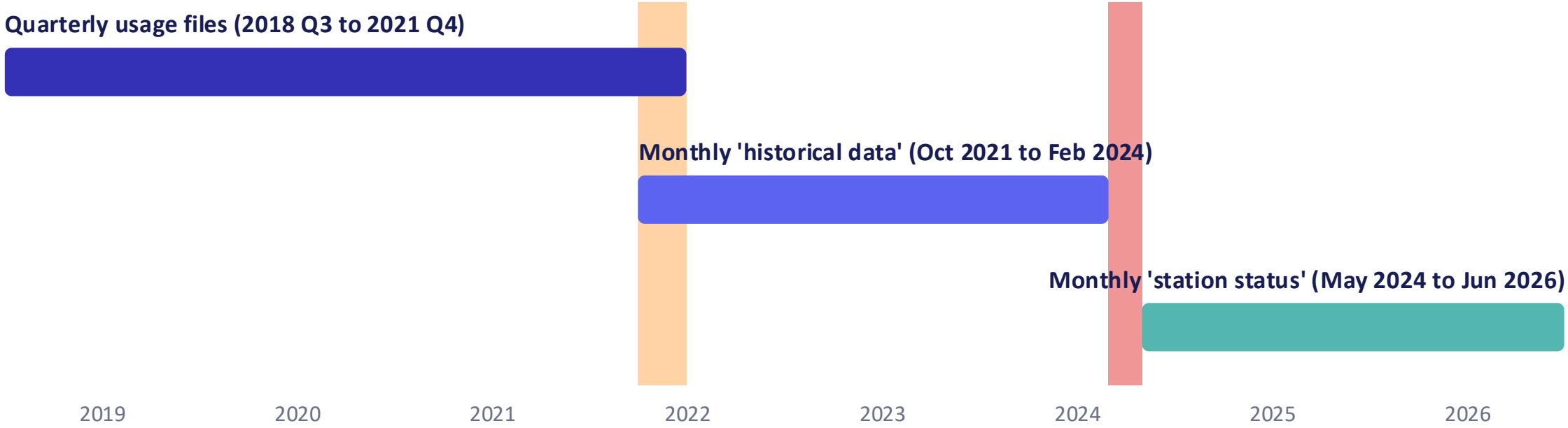
rows per monthly file

Our pipeline question

For each station and each hour: how many bikes were available, on average, at least, and at most?

A small, real, public dataset: ideal for learning, and clean enough that the mess is instructive

Real data is messy: three families of files



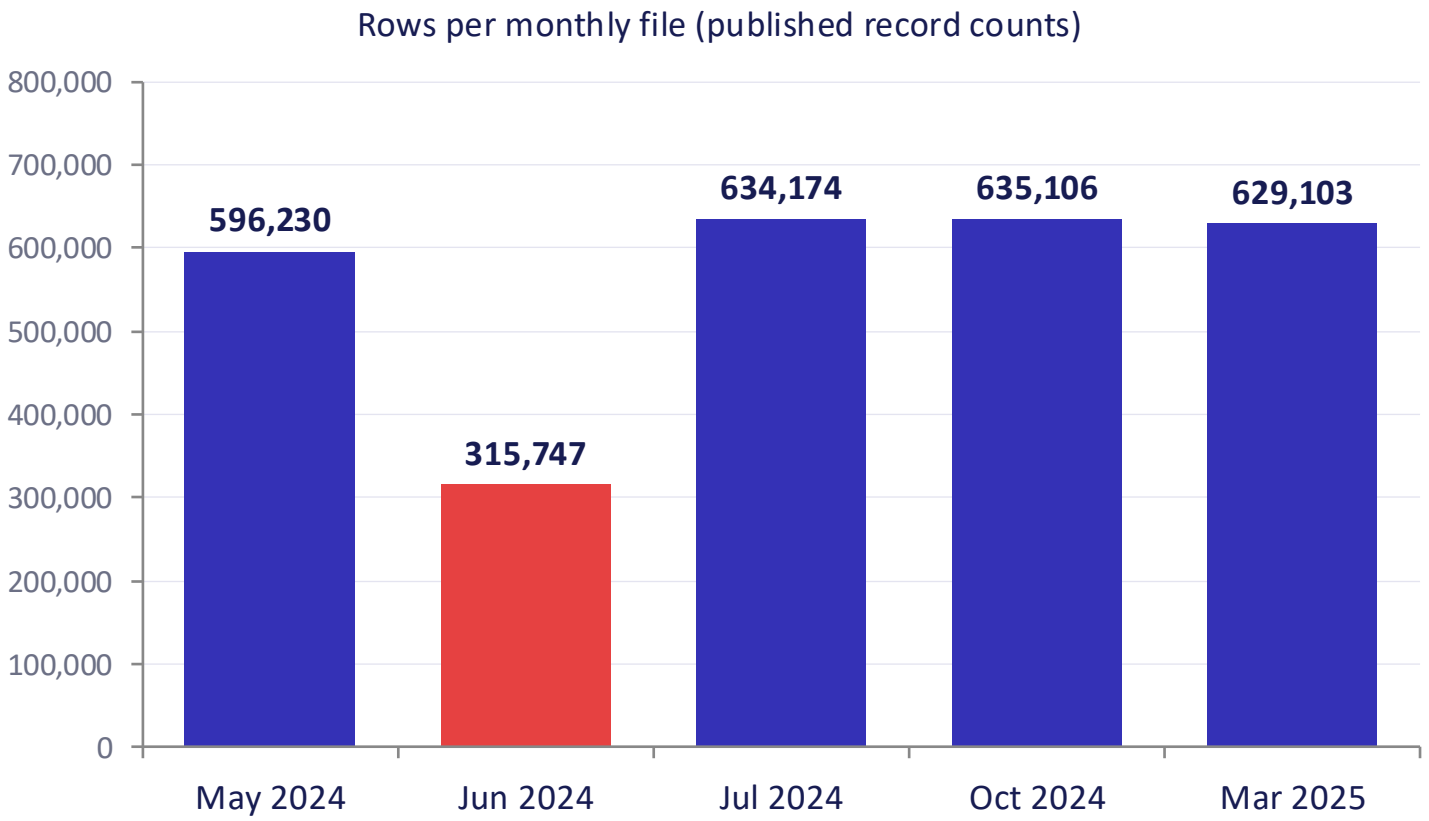
Overlap: Q4 2021 appears in a quarterly file and in three monthly files

Gap: no files for March and April 2024

The publisher warns some historical files are empty

Different shapes, a gap, a possible overlap and empty files: the pipeline must cope, not assume

June 2024 has about half the rows



What this tells us

A normal month has about 600,000 to 635,000 rows.

June 2024 has 315,747: about half.

We do not know why: an outage, a change in reporting, or missing data.

A pipeline should notice this by itself, not a person by luck.

Row counts per file are the cheapest data-quality check there is

The columns changed over time: schema drift

In the 2024 files and in the 2026 file (9)

last_reported, station_id, num_bikes_available, is_renting, name, address, lat, lon, capacity

Only in the older 2024 files (6 more)

system_id, num_docks_available, is_installed, is_returning, short_name, region_id

What our pipeline really needs: just three columns

station_id, last_reported, num_bikes_available (read by NAME, in each file, never by position)

Column lists come from the dataset's published data dictionaries for May 2024 and February 2026. Your own `head` of each file shows the truth.

Pause and think (about 30 seconds)

Scenario

You run Lecture 5's contract and cleaning rules on every Dublin Bikes file, every month, with nobody watching.

What could go wrong? Write down three things.

0:30

Think on your own. On the replay, pause the video. Then compare your list with mine.

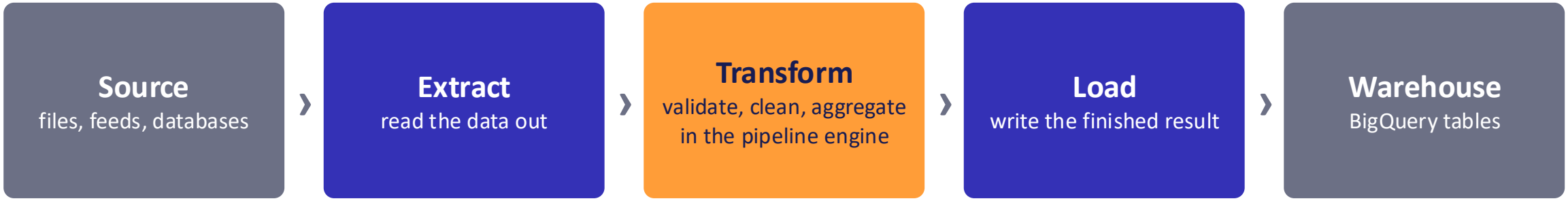
Hold your three things: the next blocks are the answers

2

ETL, ELT and good design

Where the transformation happens, and what makes a pipeline trustworthy.

ETL: transform before you load



In our demo

A Beam pipeline on Dataflow reads the CSV files, validates and aggregates them, and loads only the hourly result into BigQuery.

Good when

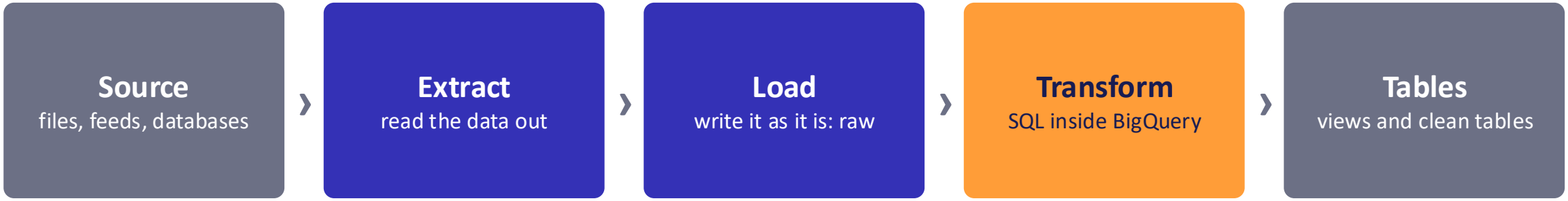
Raw data is huge or sensitive and should not all be stored.

Transformations are complex code, not just SQL.

You need streaming.

ETL: the pipeline does the heavy work, the warehouse receives clean results

ELT: load first, transform inside the warehouse



In our demo

bq load puts the raw June file into BigQuery untouched. Then a SQL query computes the same hourly table.

Good when

The warehouse is powerful and storage is cheap.

You want to keep the raw data and change your mind later.

The transformation is expressible in SQL.

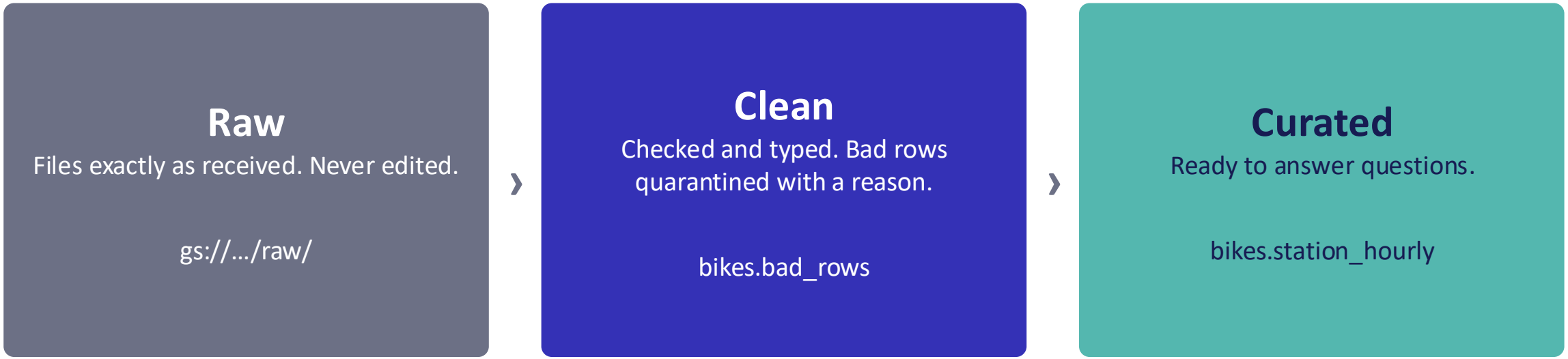
ELT: land the raw data first, then transform it with the warehouse's own power

ETL or ELT? A comparison

	ETL	ELT
Where transformation runs	In a processing engine (Beam on Dataflow)	Inside the warehouse (SQL in BigQuery)
Raw data kept?	Only if you keep the files yourself	Yes, loaded as is
Changing your mind	Change code, re-run the pipeline	Change the SQL, re-run the query
Best for	Complex logic, streaming, big raw data	Analytics, SQL-friendly reshaping
Skills needed	Programming (Python, Java)	SQL
Real systems	Usually a mix of both	Usually a mix of both

It is not a contest: we will do both in the demo, and check that they agree

Three layers: raw, clean, curated



Also called bronze, silver and gold. The idea: never lose the original, so you can always rebuild the rest.

Keep the raw layer untouched: it is your safety net when a rule turns out to be wrong

Batch or streaming? Bounded or unbounded data

Batch: bounded data

A fixed set of data with a beginning and an end.

Example: this month's CSV file.

The job starts, finishes and stops.

Cost: you pay only while it runs.

Our demo is a batch pipeline.

Streaming: unbounded data

Data that keeps arriving, with no end.

Example: the live Dublin Bikes feed, every minute.

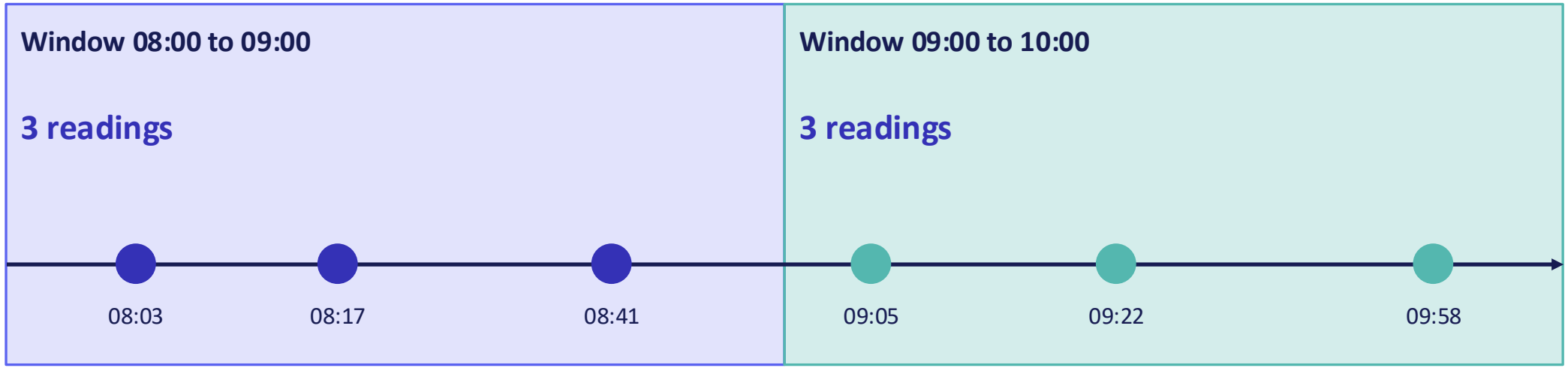
The job runs until you stop it.

Cost: it keeps running, so it keeps costing.

The same Beam code can do both.

A streaming job left running is the most expensive mistake a student can make

Event time and windows



Event time

When the bike count was true. Windows follow this.

Processing time

When our pipeline happened to see it. A late reading still counts in its event-time window.

Group by when it happened, not by when it arrived: that is what makes late data harmless

Six qualities of a trustworthy pipeline

Repeatable

Same input, same output, every time.

Idempotent

Running twice never doubles the data.

Observable

Counters, logs and metrics tell you what happened.

Testable

Parts are checked by automatic tests.

Fails loudly

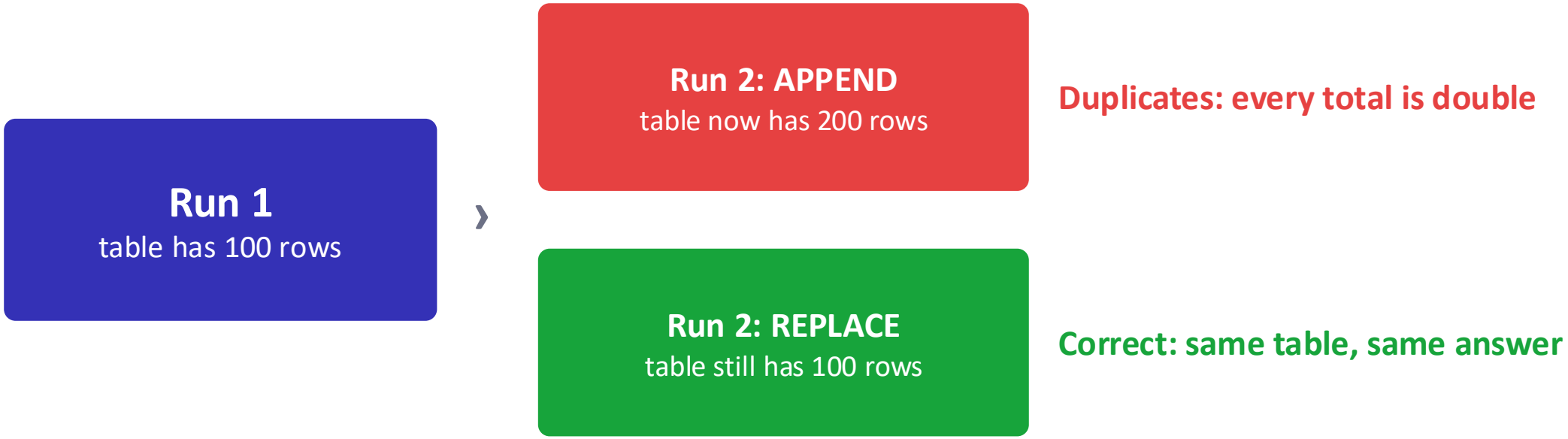
Bad data is counted and quarantined, never hidden.

Cost-aware

Caps, small machines and clean-up.

Every line of the demo exists to give the pipeline one of these six qualities

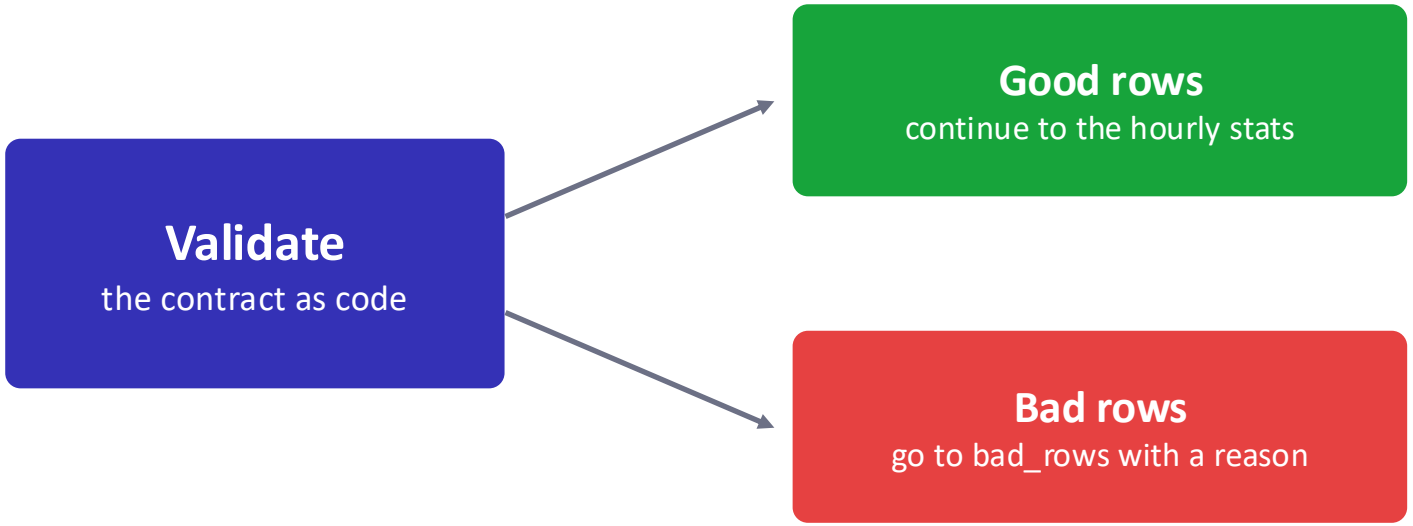
Idempotent: safe to run again



Our demo replaces the whole table on each run (WRITE_TRUNCATE). Bigger systems replace one day or month at a time, called a partition.

If a job fails halfway and you re-run it, the answer must still be right

Don't crash, don't hide: quarantine bad rows



Reason (from our local test)	What it means
negative_bikes	a count below zero
missing:station_id	a required column is empty
bad_timestamp	the time cannot be read
bad_number:num_bikes_available	not a number

Stopping the whole job for one bad row is brittle. Silently dropping it is dishonest.
Count it, keep it, and say why.

A pipeline should be tolerant of bad data and loud about it

Coping with schema drift

Strategy	What it means	In our pipeline
Read by name	Use the header of each file, not column positions	ReadCsvFile uses DictReader per file
Require only what you need	Three columns are essential; the rest is optional	the REQUIRED tuple
Quarantine, do not guess	A row missing a required value is set aside	RowError and the bad output
Count and alert	A jump in rejects means something changed	Beam counters
Version the contract	Write down which shape each period has	the data dictionary

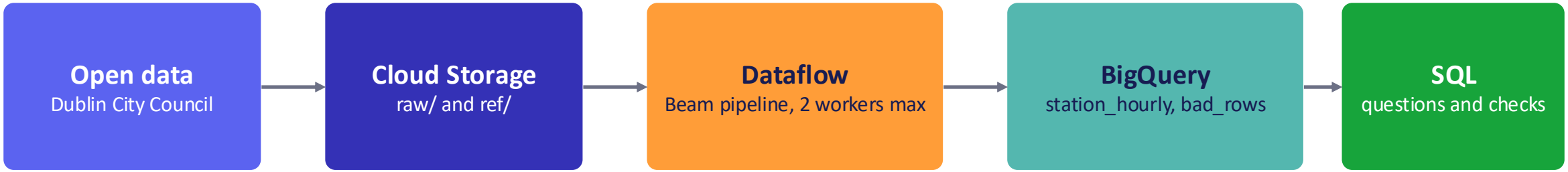
Assume the file's shape will change one day, and make that visible instead of fatal

3

Google Cloud and Apache Beam

The pieces we will use, and the vocabulary to read the code.

The architecture of our demo



ETL path (top): Beam does the transformation.



ELT path (bottom): BigQuery does the transformation.

One bucket, one dataset, one region: three Google Cloud services and two ways to transform

Apache Beam is the model; Dataflow is the engine

Apache Beam

An open-source programming model.

You write the pipeline with an SDK: Java, Python or Go.

The same code describes batch and streaming.

Google Cloud Dataflow

A fully managed service that runs Beam pipelines.

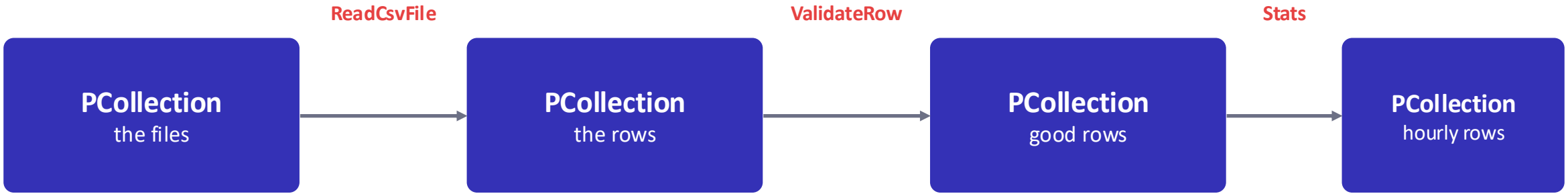
Google provides the machines, scaling and monitoring.

You pick the runner: DataflowRunner.

Runner	Where the pipeline runs	We use it for
DirectRunner	On your own machine	testing and learning, free
DataflowRunner	On Google Cloud	the real run, costs credits

Write once, test locally for free, then change one option to run in the cloud

Beam's three nouns: pipeline, PCollection, transform



Pipeline

The whole graph of steps. Nothing runs until you call run.

PCollection

A dataset in the pipeline: bounded (a file) or unbounded (a feed). It is never changed, only transformed into a new one.

PTransform

One step: a PCollection in, a PCollection out. Written with the | operator, and named with >>.

Read a Beam pipeline like a recipe: data in, step, data out, step, data out

A tiny Beam pipeline you can run right now

```
import apache_beam as beam

with beam.Pipeline() as p:
    (p
     | "Create" >> beam.Create(
         ["bike", "dock", "bike",
          "bike", "dock", "station"])
     | "Pair"   >> beam.Map(lambda word: (word, 1))
     | "Count"  >> beam.CombinePerKey(sum)
     | "Print"  >> beam.Map(print))
```

Output (the order can vary)

```
('bike', 3)
('dock', 2)
('station', 1)
```

Runs on your machine

No cloud account. This is the DirectRunner.

Create makes a PCollection. Map turns each word into (word, 1). CombinePerKey(sum) adds the ones per word. Map(print) shows the result.

Four steps, each with a name: the same shape as our Dublin Bikes pipeline, only bigger

When Map is not enough: ParDo, side inputs, tagged outputs

ParDo and DoFn

A class instead of a one-liner. It can keep counters and emit zero, one or many results per input.

```
beam.ParDo(ValidateRow())
```

Tagged outputs

One step with several named results: good rows on the main output, bad rows on another.

```
.with_outputs(BAD, main="good")
```

Side input

A small lookup table given to every worker, like station names.

```
names=beam.pvalue.AsDict(stations)
```

These three ideas are all you need to read the whole demo pipeline

Grouping, combining and windows

Stamped events
(42, 5 bikes) at 08:03
(42, 7 bikes) at 08:41
(30, 12 bikes) at 08:10
(42, 4 bikes) at 09:05

›

Grouped by key and hour
station 42, 08:00 → [5, 7]
station 30, 08:00 → [12]
station 42, 09:00 → [4]

›

Combined: one row each
42, 08:00 → 2 readings,
avg 6.0, min 5, max 7
30, 08:00 → 1 reading
42, 09:00 → 1 reading

CombinePerKey with a CombineFn

Computes partial results on many workers, then merges them. It never has to hold all of a group in memory.

FixedWindows(3600)

Cuts event time into one-hour windows, so the key becomes (station, hour) without us writing that.

Window, then key, then combine: the pattern behind almost every aggregation pipeline

What Dataflow adds



Managed

No servers to set up. Google starts workers and runs your Beam pipeline, batch or streaming.



Scales and shows you

Autoscaling, a job graph, logs and metrics for every job.



Templates

Ready-made pipelines you can start without writing Beam code, such as Pub/Sub to BigQuery.

But autoscaling is also a way to spend your credits

The default limit on workers can be very large (reported as 1,000 for batch jobs). Always set --max_num_workers yourself.

Dataflow does the operating for you; you still own the budget

Cost guardrails for the demo and the lab

Guardrail	How we do it
Batch, not streaming	The job ends by itself
Cap the workers	--max_num_workers 2
Small machines	--machine_type e2-standard-2
Small input	Two monthly files, 40 to 80 MB each
One region	Bucket, dataset and Dataflow all in europe-west1
Always clean up	cleanup.sh stops jobs and deletes the data
Measure	Read your own job's cost afterwards and write it down

$50

of credits per student, shared across the whole course

< $5

our target for this lab: a few cents for the job itself, rest is safety margin

Budget alerts only notify. Caps and clean-up are what actually protect your credits

4

The demo, line by line

Set up, understand the code, test it, run it, query it, clean up.

The demo in nine steps

0 Prepare
Project, credits, Cloud Shell

1 Set up
Services, roles, bucket, dataset

2 Get the data
Download, look, upload

3 Read the code
Eight short pieces

4 Test locally
Unit tests and a local run

5 Run on Dataflow
One command

6 Watch the job
Graph, counters, logs

7 Ask questions
ELT and SQL checks

8 Clean up
Stop and delete everything

Steps 1 to 8 are exactly what you will do in the lab, with the same scripts

Step 0: prepare your project and Cloud Shell

1

Create a project

In the Google Cloud console, open the project picker and choose New project. Note its project ID (not the name).

2

Link your credits

Billing > Link a billing account, and choose the account that holds your course credits.

3

Open Cloud Shell

Click the >_ button at the top right. It is a free terminal in your browser with gcloud already installed.

```
# 4. Get the demo kit
git clone https://github.com/AbNeil/lecture6_demo
cd lecture6_demo/lecture6_demo
chmod +x *.sh

# 5. Python environment and libraries (1-2 min)
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

The repository has a folder inside a folder, so there are two cd steps. chmod +x makes the scripts runnable, because files from GitHub are not executable by default. Do this once; later steps assume you are in this folder with the environment active.

Project ID, linked credits, Cloud Shell, kit and environment: five minutes, once

Step 1: setup_gcp.sh (1/2) billing and services

```
echo "== 1. Is billing (your course credits) linked to this project?"
BILLING=$(gcloud billing projects describe "$PROJECT_ID" --format="value(billingEnabled)")
if [ "$BILLING" != "True" ]; then
  echo "Billing is NOT enabled. In the console: Billing > Link a billing account, choose your credits account, then re-run."
  exit 1
fi

echo "== 2. Turn on the services (takes about a minute)"
gcloud services enable dataflow.googleapis.com compute.googleapis.com storage.googleapis.com \
  bigquery.googleapis.com logging.googleapis.com cloudresourcemanager.googleapis.com
```

Check billing first

gcloud billing projects describe asks Google whether credits are linked. If not, the script stops with instructions instead of failing later.

Turn on the services

A new project has almost everything switched off. We enable Dataflow, Compute (the workers), Storage, BigQuery and Logging.

Safe to re-run

set -euo pipefail stops at the first error, and every step can be repeated without harm.

Step 1: setup_gcp.sh (2/2) roles, bucket, dataset

```
echo "== 3. Give the Dataflow worker account the roles it needs"
PROJECT_NUMBER=$(gcloud projects describe "$PROJECT_ID" --format="value(projectNumber)")
WORKER_SA="${PROJECT_NUMBER}-compute@developer.gserviceaccount.com"
until gcloud iam service-accounts describe "$WORKER_SA" >/dev/null 2>&1; do
  echo "   waiting for the default worker account to appear..."; sleep 5
done
for ROLE in roles/dataflow.worker roles/storage.objectAdmin roles/bigquery.dataEditor roles/bigquery.jobUser; do
  gcloud projects add-iam-policy-binding "$PROJECT_ID" \
    --member="serviceAccount:${WORKER_SA}" --role="$ROLE" --condition=None --quiet >/dev/null
  echo "   granted $ROLE"
done

echo "== 4. Create the bucket and the BigQuery dataset (same region)"
gcloud storage buckets describe "gs://$BUCKET" >/dev/null 2>&1 || \
  gcloud storage buckets create "gs://$BUCKET" --location="$REGION" --uniform-bucket-level-access
bq --location="$REGION" show "$PROJECT_ID:$DATASET" >/dev/null 2>&1 || \
  bq --location="$REGION" mk --dataset "$PROJECT_ID:$DATASET"
```

Worker account roles

Dataflow workers run as a service account. It needs permission to work (dataflow.worker), use the bucket, and write BigQuery.

Same region everywhere

Bucket and dataset are both in europe-west1, the same as the job. Mismatched locations are a classic failure.

env.sh

The script saves PROJECT_ID, REGION, BUCKET and DATASET so every later script just reads them.

Step 2: get_data.sh: download, look, upload

```
mkdir -p data
curl -fL --retry 3 -o data/dublin-bikes_station_status_062024.csv "$JUN2024"
curl -fL --retry 3 -o data/dublin-bikes_station_status_022026.csv "$FEB2026"
curl -fL --retry 3 -o data/dublin.csv "$STATIONS"

echo; echo "== File sizes"; ls -lh data
echo; echo "== LOOK AT THE DATA FIRST: the header and two rows of each file"
for f in data/dublin-bikes_station_status_*.csv data/dublin.csv; do
  echo "--- $f"; head -n 3 "$f"
done
echo; echo "== Rows (including the header line)"; wc -l data/*.csv

gcloud storage cp data/dublin-bikes_station_status_*.csv "gs://$BUCKET/raw/"
gcloud storage cp data/dublin.csv "gs://$BUCKET/ref/"
echo; gcloud storage ls -l "gs://$BUCKET/raw/" "gs://$BUCKET/ref/"
```

Three downloads

June 2024 and February 2026 status files, and the station list. The URLs are variables at the top of the script.

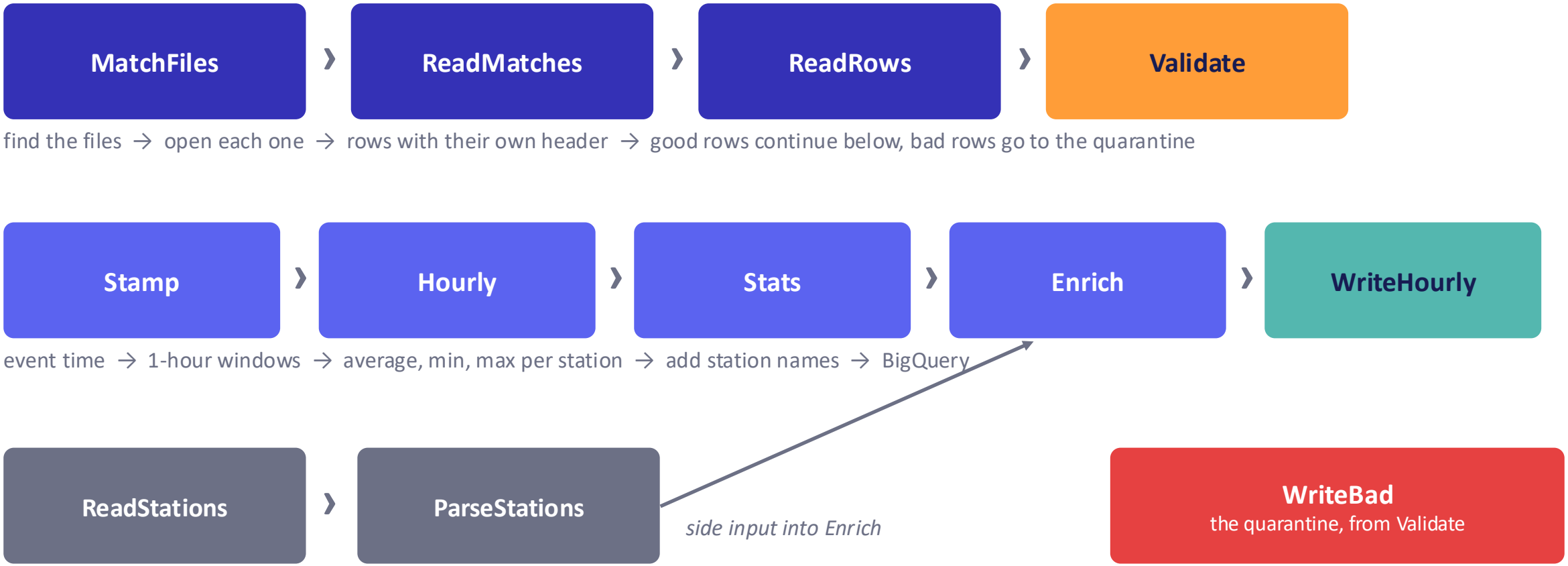
Look before you run

head -n 3 prints each file's header and two rows. This is where you see the real columns and the timestamp format.

Upload to the raw layer

Status files go to raw/, the station list to ref/. We never edit them again.

The shape of the pipeline we will read



Nine named steps. Every name you see here is a label in the Dataflow job graph later

Code 1: imports and two constants

```python
import argparse
import csv
import io
import json
import logging
from datetime import datetime, timezone

import apache_beam as beam
from apache_beam.io import fileio
from apache_beam.metrics import Metrics
from apache_beam.options.pipeline_options import GoogleCloudOptions
from apache_beam.options.pipeline_options import PipelineOptions
from apache_beam.transforms import window

REQUIRED = ("station_id", "last_reported", "num_bikes_available")
BAD = "bad"  # name of the quarantine (dead-letter) output
```

What to notice

1. Standard library first: csv, io and json read and write text; datetime handles times.
2. apache_beam is the library. fileio finds and opens files; Metrics creates counters; window gives us time windows.
3. REQUIRED lists the only three columns the pipeline cannot live without.
4. BAD is just a name. It labels the second output of the validation step.

Code 2: ReadCsvFile reads one file with its own header

```
class ReadCsvFile(beam.DoFn):
    """One element in = one file. We read the header of THAT file, so
files
    with different columns (schema drift) are all handled correctly."""

    def __init__(self):
        self.files = Metrics.counter("bikes", "files_read")
        self.empty = Metrics.counter("bikes", "empty_files")
        self.rows = Metrics.counter("bikes", "rows_read")

    def process(self, readable_file):
        name = readable_file.metadata.path.rsplit("/", 1)[-1]
        self.files.inc()
        count = 0
        with readable_file.open() as raw:
            text = io.TextIOWrapper(raw, encoding="utf-8-sig", newline="")
            for row in csv.DictReader(text):
                count += 1
                row["_source"] = name
                yield row
        self.rows.inc(count)
        if count == 0:
            self.empty.inc()
```

What to notice

1. A DoFn is a class with a process method. One input in, any number of outputs out.
2. The input is a whole file, not a row. We read its header with DictReader, so each file is understood on its own terms.
3. utf-8-sig quietly removes an invisible marker some CSV files start with.
4. yield sends each row onward with its file name added, so we can trace problems later.
5. Counters count files, rows, and empty files. An empty file gives zero rows.

Code 3: reading times and numbers safely

```
class RowError(Exception):
    """Raised when a row breaks the data contract; the message is the reason."""

def parse_timestamp(value):
    """Return seconds since 1970 (UTC). Accepts epoch seconds or ISO text."""
    text = (value or "").strip()
    try:
        return float(text)
    except ValueError:
        pass
    try:
        parsed = datetime.fromisoformat(text.replace("Z", "+00:00"))
    except ValueError:
        raise RowError("bad_timestamp")
    if parsed.tzinfo is None:
        parsed = parsed.replace(tzinfo=timezone.utc)
    return parsed.timestamp()

def to_int(row, column):
    try:
        return int(float(row[column]))
    except (ValueError, TypeError):
        raise RowError("bad_number:" + column)
```

What to notice

1. RowError is our own exception. Its message is the reason a row is rejected.
2. parse_timestamp tries a number of seconds first (the GBFS standard), then ISO text such as 2024-06-01 08:00:07+00:00 or ...Z.
3. No time zone? We assume UTC, so every timestamp means the same thing.
4. to_int accepts 12 and 12.0. Anything else becomes a bad_number reason.

Code 4: ValidateRow (1/2) the contract as code

```
class ValidateRow(beam.DoFn):
    """The data contract from Lecture 5, as code. Good rows go to the main
    output, bad rows go to the BAD output together with the reason."""

    def __init__(self):
        self.good = Metrics.counter("bikes", "good_rows")
        self.bad = Metrics.counter("bikes", "bad_rows")
        self.over = Metrics.counter("bikes", "over_capacity")

    def process(self, row):
        try:
            missing = [c for c in REQUIRED if row.get(c) in (None, "")]
            if missing:
                raise RowError("missing:" + ",".join(missing))
            station_id = to_int(row, "station_id")
            bikes = to_int(row, "num_bikes_available")
            seconds = parse_timestamp(row["last_reported"])
            if bikes < 0:
                raise RowError("negative_bikes")
```

What to notice

1. This is Lecture 5's data contract, now a class that runs on every row.
2. Three counters: good rows, bad rows, and a plausibility warning.
3. First, check that the three required columns are present and not empty.
4. Then convert station and bike count to integers and the time to seconds.
5. A negative bike count is impossible, so we raise RowError.

Code 5: ValidateRow (2/2) and stamp

```
        except RowError as err:
            self.bad.inc()
            yield beam.pvalue.TaggedOutput(BAD, {
                "reason": str(err),
                "source": row.get("_source"),
                "raw": json.dumps(row)[:500]})
            return
        capacity = row.get("capacity")
        if capacity not in (None, "") and bikes > float(capacity):
            self.over.inc()  # odd but possible: keep the row, count it
        self.good.inc()
        yield {"station_id": station_id, "bikes": bikes, "seconds": seconds}


def stamp(rec):
    """Give each element its real event time: windows follow the data."""
    return window.TimestampedValue(
        (rec["station_id"], rec["bikes"]), rec["seconds"])
```

What to notice

1. except catches a bad row. We count it, then yield it on the BAD output with the reason, the file name and the raw row.
2. return means: do not send this row onward as good.
3. More bikes than capacity is odd but possible. Lecture 5's rule: flag, do not delete. We keep the row and count it.
4. stamp gives each reading its event time, so windows follow when it happened, not when we read it.

Code 6: BikeStats combines readings in parallel

```python
class BikeStats(beam.CombineFn):
    """Count, average, min and max of bikes available, computed in parallel."""

    def create_accumulator(self):
        return (0, 0, None, None)  # n, total, lowest, highest

    def add_input(self, acc, bikes):
        n, total, low, high = acc
        return (n + 1, total + bikes,
                bikes if low is None else min(low, bikes),
                bikes if high is None else max(high, bikes))

    def merge_accumulators(self, accs):
        n, total, low, high = 0, 0, None, None
        for a_n, a_total, a_low, a_high in accs:
            n, total = n + a_n, total + a_total
            if a_low is not None:
                low = a_low if low is None else min(low, a_low)
                high = a_high if high is None else max(high, a_high)
        return (n, total, low, high)

    def extract_output(self, acc):
        n, total, low, high = acc
        return {"samples": n, "min_bikes": low, "max_bikes": high,
                "avg_bikes": round(total / n, 2) if n else None}
```

What to notice

1. A CombineFn does an aggregation in four small steps so many workers can share the job.
2. create_accumulator starts empty. add_input folds in one reading.
3. merge_accumulators joins the partial results from different workers.
4. extract_output produces the final answer: samples, minimum, maximum and average.
5. We carry the total and count, not the average itself, because averages cannot be merged directly.

Code 7: station names as a side input

```
def parse_station(line):
    """Reference line -> (station number, name). Bad lines are skipped."""
    try:
        cells = next(csv.reader([line]))
        return [(int(cells[0]), cells[1])]
    except (ValueError, IndexError, StopIteration):
        return []


class ToRow(beam.DoFn):
    """Make an output row; station names arrive as a side input."""

    def __init__(self):
        self.unknown = Metrics.counter("bikes", "unknown_station")

    def process(self, element, win=beam.DoFn.WindowParam, names=None):
        station_id, stats = element
        name = names.get(station_id)
        if name is None:
            self.unknown.inc()
        start = datetime.fromtimestamp(win.start.micros / 1e6, tz=timezone.utc)
        yield {"station_id": station_id, "station_name": name,
               "hour_start": start.strftime("%Y-%m-%d %H:%M:%S UTC"), **stats}
```

What to notice

1. parse_station turns one line of the reference file into (number, name). A bad line returns an empty list and is skipped.
2. ToRow receives the aggregated result and the whole station table as names.
3. names.get looks the station up. A station we do not know is counted, not dropped.
4. WindowParam gives the window the result belongs to, so we can write the hour it starts.

Code 8: build() wires the whole pipeline

```
def build(p, args):
    """Wire the pipeline: read, validate, window, aggregate, enrich."""
    stations = (
        p | "ReadStations" >> beam.io.ReadFromText(
            args.stations, skip_header_lines=1)
          | "ParseStations" >> beam.FlatMap(parse_station))

    checked = (
        p | "Match" >> fileio.MatchFiles(args.input)
          | "OpenFiles" >> fileio.ReadMatches()
          | "ReadRows" >> beam.ParDo(ReadCsvFile())
          | "Validate" >> beam.ParDo(ValidateRow()).with_outputs(
              BAD, main="good"))

    hourly = (
        checked.good
        | "Stamp" >> beam.Map(stamp)
        | "Hourly" >> beam.WindowInto(window.FixedWindows(3600))
        | "Stats" >> beam.CombinePerKey(BikeStats())
        | "Enrich" >> beam.ParDo(
            ToRow(), names=beam.pvalue.AsDict(stations)))
    return hourly, checked[BAD]
```

What to notice

1. Each step is one line: | name >> transform. The names appear in the Dataflow job graph.
2. stations is a small separate branch: read the reference file and parse it.
3. checked finds files, opens them, reads rows and validates them, with two outputs.
4. checked.good continues: stamp, one-hour windows, stats, and enrich with the stations.
5. build() returns the two results. Writing them is a separate function.

Code 9: write_results sends the output to its destination

```
def write_results(hourly, bad, args, project):
    """Send the two results to BigQuery or to local files."""
    if args.bq_dataset:
        def sink(table, schema):
            return beam.io.WriteToBigQuery(
                f"{project}:{args.bq_dataset}.{table}", schema=schema,
                create_disposition=beam.io.BigQueryDisposition.CREATE_IF_NEEDED,
                # replace, never append: re-running gives the same table
                write_disposition=beam.io.BigQueryDisposition.WRITE_TRUNCATE,
                method=beam.io.WriteToBigQuery.Method.FILE_LOADS)
        hourly | "WriteHourly" >> sink("station_hourly", HOURLY_SCHEMA)
        bad | "WriteBad" >> sink("bad_rows", BAD_SCHEMA)
    else:
        text = beam.io.WriteToText
        (hourly | "HourlyJson" >> beam.Map(json.dumps)
                | "WriteHourly" >> text(args.output, file_name_suffix=".jsonl"))
        (bad | "BadJson" >> beam.Map(json.dumps)
             | "WriteBad" >> text(args.output + "_bad",
                                  file_name_suffix=".jsonl"))
```

What to notice

1. One pipeline, two destinations: BigQuery when --bq_dataset is given, local files otherwise.
2. WRITE_TRUNCATE replaces the table on every run, so re-running never duplicates rows.
3. CREATE_IF_NEEDED creates the table from the schema string the first time.
4. FILE_LOADS writes batch results through temporary files in your bucket: that is what --temp_location is for.

Code 10: run() ties it together

```python
def run(argv=None):
    args, beam_args = parse_args(argv)
    options = PipelineOptions(beam_args, save_main_session=True)
    project = options.view_as(GoogleCloudOptions).project
    local = options.get_all_options().get("runner") in (None, "DirectRunner")
    if local:
        logging.getLogger().setLevel(logging.WARNING)  # short local output
        logging.getLogger("apache_beam.utils.subprocess_server").setLevel(
            logging.ERROR)

    pipeline = beam.Pipeline(options=options)
    hourly, bad = build(pipeline, args)
    write_results(hourly, bad, args, project)
    result = pipeline.run()
    result.wait_until_finish()
    if local:
        print_counters(result)
```

What to notice

1. parse_args reads our options; everything else (runner, project, region, workers) goes to Beam as pipeline options.
2. save_main_session lets workers use the classes defined in this file.
3. The pipeline is built, written, run, and we wait for it to finish.
4. On a local run we print the counters at the end: that is our test report.
5. The same file runs locally or on Dataflow. Only the options change.

Step 4a: unit tests

```
GOOD = {"station_id": "42", "num_bikes_available": "5", "capacity": "20",
        "last_reported": "2024-06-01 08:00:07+00:00", "_source": "a.csv"}


def test_validate_sends_good_and_bad_rows_to_the_right_place():
    rows = [GOOD,
            dict(GOOD, num_bikes_available="-1"),
            dict(GOOD, station_id=""),
            dict(GOOD, last_reported="nope"),
            dict(GOOD, num_bikes_available="many")]
    reasons = ["negative_bikes", "missing:station_id", "bad_timestamp",
               "bad_number:num_bikes_available"]
    with TestPipeline() as p:
        out = (p | beam.Create(rows)
                 | beam.ParDo(bp.ValidateRow()).with_outputs(
                     bp.BAD, main="good"))
        count = out.good | beam.combiners.Count.Globally()
        assert_that(count, equal_to([1]), label="one good row")
        why = out[bp.BAD] | beam.Map(lambda row: row["reason"])
        assert_that(why, equal_to(reasons), label="four reasons")


def test_bike_stats_combines_correctly():
    readings = [(1, 4), (1, 8), (1, 0), (2, 5)]
    with TestPipeline() as p:
        out = (p | beam.Create(readings)
                 | beam.CombinePerKey(bp.BikeStats()))
        assert_that(out, equal_to([
            (1, {"samples": 3, "avg_bikes": 4.0,
                 "min_bikes": 0, "max_bikes": 8}),
            (2, {"samples": 1, "avg_bikes": 5.0,
                 "min_bikes": 5, "max_bikes": 5})]))
```

python -m pytest -q test_pipeline.py

1. Pipelines are code, so we test the pieces with small known inputs.
2. Validation is tested with one good row and four bad ones; we check each reason.
3. BikeStats is tested on three readings for one station and one for another.
4. Expected result: 5 passed. If a test fails, the pipeline is not ready.

Step 4b: run it locally on the sample

```
python make_sample_data.py
python bike_pipeline.py --runner DirectRunner \
  --input "sample/raw/*.csv" \
  --stations sample/ref/stations.csv \
  --output out/hourly
```

Counter	What it tells us
files_read 3, empty_files 1	three files, one header-only
rows_read 186 = 182 + 4	every row is accounted for
bad_rows 4	one of each broken kind
over_capacity 1, unknown_station 1	odd rows kept, but counted

Counters (real output)

```
bad_rows         4
empty_files      1
files_read       3
good_rows        182
over_capacity    1
rows_read        186
unknown_station  1
```

Result files

31 hourly rows in out/hourly-*.jsonl and 4 quarantined rows in out/hourly_bad-*.jsonl.

186 rows in, 182 good plus 4 bad out: nothing vanished, and it takes seconds, not minutes

What rehearsal caught: the wrong file in the glob

My first run: --input "sample/*.csv"

The pattern also matched stations.csv, the reference file, which has different columns.

The fix: separate folders

Status files in raw/, the reference in ref/. Input is sample/raw/*.csv.

Counters before the fix

```
bad_rows        9
files_read      4
rows_read       191
```

Counters after the fix

```
bad_rows        4
files_read      3
rows_read       186
```

Counters and the quarantine caught it in seconds: that is what they are for

Step 5: run_dataflow.sh runs the same pipeline on Dataflow

```
python bike_pipeline.py \
  --runner DataflowRunner \
  --project "$PROJECT_ID" \
  --region "$REGION" \
  --temp_location "gs://$BUCKET/tmp" \
  --staging_location "gs://$BUCKET/staging" \
  --job_name "bikes-$(date +%Y%m%d-%H%M%S)" \
  --max_num_workers 2 \
  --machine_type e2-standard-2 \
  --input "gs://$BUCKET/raw/*.csv" \
  --stations "gs://$BUCKET/ref/dublin.csv" \
  --bq_dataset "$DATASET"
```

Option	Meaning
--runner DataflowRunner	run in the cloud, not locally
--project, --region	where the job lives
--temp_location	bucket folder for temporary files
--job_name	unique name with the time
--max_num_workers 2	cost cap: at most two workers
--machine_type e2-standard-2	small machines
--input, --stations	files in Cloud Storage
--bq_dataset	write to BigQuery, not files

Step 6: watch the job in the console

Open Dataflow > Jobs

Find your job by name. States you will see: queued, running, then succeeded or failed.

Job graph

Each box is one of our named steps: ReadRows, Validate, Stats, WriteHourly. Click one to see how many elements passed through.

Job info and metrics

Region, number of workers, and the custom counters we coded: good_rows, bad_rows and the rest.

Logs

If a job fails, start here. The first error message usually names the problem.

Start-up takes a few minutes before any work happens: that is normal. Layout of the console changes over time; follow the step names you coded.

Step 7a: ELT load, then ask questions with SQL

```
echo "== ELT: load the raw June 2024 file into BigQuery as
it is"
bq load --source_format=CSV --skip_leading_rows=1 --
autodetect --replace \
  "$DATASET.raw_2024_06" "gs://$BUCKET/raw/dublin-
bikes_station_status_062024.csv"
```

```
-- ## 1. Did the pipeline load anything? (hourly rows,
stations, first and last hour)
SELECT COUNT(*) AS hourly_rows,
       COUNT(DISTINCT station_id) AS stations,
       MIN(hour_start) AS first_hour,
       MAX(hour_start) AS last_hour
FROM `YOUR_PROJECT.bikes.station_hourly`;

-- ## 2. What did the quarantine catch? (bad rows by file
and reason)
SELECT source, reason, COUNT(*) AS bad_rows
FROM `YOUR_PROJECT.bikes.bad_rows`
GROUP BY source, reason
ORDER BY bad_rows DESC;
```

What each part does

bq load puts the raw June file into BigQuery untouched. --autodetect guesses the columns, --replace makes it safe to re-run.

Query 1: did anything load? Rows, stations, first and last hour.

Query 2: what did the quarantine catch? Bad rows grouped by file and reason.

If last_reported loads as text, SAFE_CAST in the later queries still turns it into a timestamp.

Step 7b: do ETL and ELT agree? A reconciliation

```
WITH elt AS (
  SELECT station_id,
         TIMESTAMP_TRUNC(SAFE_CAST(last_reported AS TIMESTAMP), HOUR) AS hour_start,
         COUNT(*) AS samples,
         ROUND(AVG(num_bikes_available), 2) AS avg_bikes
  FROM `YOUR_PROJECT.bikes.raw_2024_06`
  WHERE station_id IS NOT NULL
    AND num_bikes_available >= 0
    AND SAFE_CAST(last_reported AS TIMESTAMP) >= TIMESTAMP '2024-06-01'
    AND SAFE_CAST(last_reported AS TIMESTAMP) <  TIMESTAMP '2024-07-01'
  GROUP BY station_id, hour_start
),
etl AS (
  SELECT station_id, hour_start, samples, avg_bikes
  FROM `YOUR_PROJECT.bikes.station_hourly`
  WHERE hour_start >= TIMESTAMP '2024-06-01' AND hour_start < TIMESTAMP '2024-07-01'
)
SELECT (SELECT COUNT(*) FROM elt) AS elt_rows,
       (SELECT COUNT(*) FROM etl) AS etl_rows,
       (SELECT COUNT(*) FROM elt JOIN etl USING (station_id, hour_start)
         WHERE elt.samples = etl.samples AND ABS(elt.avg_bikes - etl.avg_bikes) < 0.01) AS
matching_rows;
```

How to read it

1. elt: the hourly numbers computed by SQL directly on the raw June table.
2. etl: the same numbers from our Beam pipeline's output, June only.
3. The last line counts hours where both give the same sample count and nearly the same average.
4. If matching_rows equals both row counts, two different methods agree: strong evidence the pipeline is right.

Step 8: cleanup.sh: stop, delete, check

```
echo "== Cancel any Dataflow job that is still running"
for JOB in $(gcloud dataflow jobs list --region="$REGION" --status=active --format="value(JOB_ID)"); do
  gcloud dataflow jobs cancel "$JOB" --region="$REGION"
done
echo "== Delete the bucket and the BigQuery dataset"
gcloud storage rm --recursive "gs://$BUCKET"
bq rm -r -f -d "$PROJECT_ID:$DATASET"
echo "== Check: these should show nothing"
gcloud dataflow jobs list --region="$REGION" --status=active
gcloud storage ls
bq ls
```

Cancel running jobs

Lists active Dataflow jobs in the region and cancels each one. A forgotten job is the main way credits disappear.

Delete the data

Removes the bucket and the BigQuery dataset, so nothing is stored or billed.

Or delete the project

gcloud projects delete removes everything at once and can be undone for 30 days.

If something goes wrong

Symptom	Likely cause	What to do
Setup says billing is not enabled	Credits not linked	Billing > Link a billing account, re-run
403 permission denied from the job	A role is missing on the worker account	Re-run setup_gcp.sh
Job will not start: quota	Region is busy or limited	Wait, or use --machine_type n1-standard-2
A download fails	The link changed	Copy the link from the dataset page
station_hourly is empty	Rows were all rejected	Query bad_rows: the reasons say why
bad_rows is huge	The glob matched a wrong file	gcloud storage ls the raw/ folder
Dataset not found in location	Mismatched regions	Keep bucket, dataset, job in europe-west1

Almost every failure leaves a clue: in the counters, the bad_rows table, or the job's first error log

Streaming: the same model, a different input

Pub/Sub topic
NYC taxi rides, public

›

Dataflow template
Pub/Sub to BigQuery, no code

›

BigQuery table
rows keep arriving

What it shows

Google's own quickstart uses the public topic projects/pubsub-public-data/topics/taxirides-realtime.

A ready-made template needs no Beam code.

The job never finishes by itself.

Lecturer demo only

A streaming job runs, and costs, until it is stopped. We show it, watch it scale, then stop it straight away. You do not run it in the lab.

Same Beam model for streaming: but a job that never ends needs a person who ends it

Orchestration: who starts the pipeline?



Scheduling

Run every night or month, or when a file lands. Cloud Scheduler is one Google Cloud tool for time triggers.

Order and retries

Step two waits for step one. A failed step is retried, then reported. Workflows and Cloud Composer (managed Airflow) do this.

Why idempotent matters here

Retries and re-runs are normal in production. Only idempotent jobs survive them.

Tool names are Google Cloud options in general; check the current documentation before choosing.

Operating a pipeline

Monitor

Job state, duration and our counters, every run.

Check the data

Rows per file and reject rate. June 2024's half-size file would be caught here.

Alert

Tell a person when a job fails or rejects spike.

Test

Unit tests run automatically on every change.

Watch the bill

Budget alerts, worker caps, and a review of each job's cost.

Re-run safely

Fix the rule, re-run from the untouched raw layer.

Building the pipeline is the beginning: most of the work is keeping it trustworthy

From Lecture 5's decisions to pipeline rules

Lecture 5 decision	Where it lives in the pipeline
Data contract	ValidateRow and the REQUIRED columns
Policy for missing and invalid values	Quarantine with a reason, and counters
Outliers: flag, do not delete	over_capacity counted, the row is kept
Keys and duplicates	station_id as key; WRITE_TRUNCATE for safe re-runs
Data dictionary	The schema strings and the README
Split first, learn from training only	Keep the raw layer; fit models downstream
Evidence for every action	Counters, the bad_rows table, the _source file name

A good EDA writes the rules; a good pipeline enforces them and shows its evidence

Summary: six ideas to keep

1

Pipelines repeat decisions

Collect, validate, transform, load, repeat: automatically.

2

ETL or ELT

Where does the transformation run: in the engine or in the warehouse?

3

Design for re-runs

Idempotent loads, a quarantine, counters, a raw layer.

4

Beam and Dataflow

Beam is the model; Dataflow is the managed engine that runs it.

5

Test, then run capped

Local first, free. Then Dataflow with workers capped.

6

Measure and clean up

Read the cost, check the reconciliation, delete everything.

A pipeline you can re-run, test, watch and afford is a pipeline you can trust

In the lab: you run this yourself

You will

Use the same demo kit.

Run and test the pipeline locally first.

Set up your own project, bucket and dataset.

Run the Dataflow job, capped at two workers.

Read the counters and check the reconciliation.

Note what your job cost, then clean up.

Ground rules

Credits linked before you start.

Batch jobs only. No streaming.

Never remove the worker cap.

Run cleanup.sh before you leave.

If something fails, read the README's troubleshooting table first.

Everything you saw today is in the kit: nothing is hidden, and nothing is magic

Reading list and links

Apache Beam programming guide	beam.apache.org/documentation/programming-guide
Google Cloud Dataflow documentation	cloud.google.com/dataflow/docs (overview and pipeline options)
Dublinbikes API dataset	data.smartdublin.ie/dataset/dublinbikes-api (CC BY 4.0)
Akidau et al. (2015)	The Dataflow Model. Proceedings of the VLDB Endowment, 8(12)
Akidau, Chernyak and Lax (2018)	Streaming Systems. O'Reilly
Kleppmann (2017)	Designing Data-Intensive Applications. O'Reilly
Reis and Housley (2022)	Fundamentals of Data Engineering. O'Reilly

Start with the Beam programming guide and the Dataflow overview page